

字符串

Scintilla

Tsinghua University

2026 年 5 月 4 日

1 记号

2 border 相关

3 ACAM

4 回文串相关

5 Z 函数

6 后缀数据结构

7 基本子串结构

8 References

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。
- 对于字符串 s ，设 $\text{pre}(s, i)$ 表示字符串 $s[1]s[2]\cdots s[i]$ ，即 s 的长度为 i 的前缀； $\text{suf}(s, i)$ 表示字符串 $s[n-i+1]\cdots s[n-1]s[n]$ ，即 s 的长度为 i 的后缀。

记号

- 设字符集为 Σ 。若无特殊说明，一般认为 Σ 为 26 个小写英文字母。
- 对于字符串 s ，设 $|s|$ 表示其长度。本文中若无特殊说明，一般用 n 代指 $|s|$ ，用 m 指代 $|t|$ ，并且默认 n, m 同阶。
- 对于字符串 s ， $s[i]$ 表示 s 的第 i 个字符， $s[i..j]$ 表示字符串 $s[i]s[i+1]\cdots s[j]$ ，下标从 1 开始。
- 对于字符串 s ，设 $\text{pre}(s, i)$ 表示字符串 $s[1]s[2]\cdots s[i]$ ，即 s 的长度为 i 的前缀； $\text{suf}(s, i)$ 表示字符串 $s[n-i+1]\cdots s[n-1]s[n]$ ，即 s 的长度为 i 的后缀。
- 对于字符串 s, t ，设 st 表示其拼接，即 $st = s[1]s[2]\cdots s[n]t[1]t[2]\cdots t[m]$ 。定义 $s^1 = s, s^i = s^{i-1}s, s^R = s[n]s[n-1]\cdots s[1]$ 。

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p]$, $s[i] = s[i + p]$ 。

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p]$, $s[i] = s[i + p]$ 。

对于 $p \in [1, n)$, 称 $\text{pre}(s, p)$ 是 s 的 border, 当且仅当 $\text{pre}(s, p) = \text{suf}(s, p)$ 。

定义

对于 $p \in [1, n]$, 称 p 是 s 的 period, 当且仅当 $\forall i \in [1, n - p]$, $s[i] = s[i + p]$ 。

对于 $p \in [1, n]$, 称 $\text{pre}(s, p)$ 是 s 的 border, 当且仅当 $\text{pre}(s, p) = \text{suf}(s, p)$ 。

不难发现, $\text{pre}(s, p)$ 是 s 的 border 当且仅当 $n - p$ 是 s 的 period。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

这个数组通过增量法求出。算法维护一个指针 j ，表示当前字符串的最长 border 长度。每次新加入一个字符时尝试能否继续匹配，若不能则令 $j \leftarrow \text{fail}[j]$ ，直至能够匹配或 $j = 0$ 。

KMP

KMP 算法能够在 $\mathcal{O}(n)$ 的时间内求出 s 的所有 border。

具体地，KMP 算法求出了一个长度为 n 的数组 fail ， fail_i 表示 $\text{pre}(s, i)$ 的最长 border 长度。

这个数组通过增量法求出。算法维护一个指针 j ，表示当前字符串的最长 border 长度。每次新加入一个字符时尝试能否继续匹配，若不能则令 $j \leftarrow \text{fail}[j]$ ，直至能够匹配或 $j = 0$ 。

设 $\mathbf{Bd}(i)$ 表示 $\text{pre}(s, i)$ 的所有 border。若 $\text{fail}[i] = 0$ ，那么 $\mathbf{Bd}(i) = \emptyset$ ；否则

$$\mathbf{Bd}(i) = \{\text{fail}[i]\} \cup \mathbf{Bd}(\text{fail}[i]).$$

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

Proof

不妨设 $p < q$ 。对于任意 $1 \leq i \leq n - q + p$, 因为 $i - p \geq 1$ 和 $i + q \leq n$ 至少有一个满足, 可以通过 $s[i - p]$ 或 $s[i + q]$ 推出 $s[i] = s[i - p + q]$, 进而 $q - p$ 也是 s 的 period。

Periodicity Lemma

Theorem 1.1 (Weak Periodicity Lemma)

若 $p + q \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

Proof

不妨设 $p < q$ 。对于任意 $1 \leq i \leq n - q + p$, 因为 $i - p \geq 1$ 和 $i + q \leq n$ 至少有一个满足, 可以通过 $s[i - p]$ 或 $s[i + q]$ 推出 $s[i] = s[i - p + q]$, 进而 $q - p$ 也是 s 的 period。

Theorem 1.2 (Periodicity Lemma)

若 $p + q - \gcd(p, q) \leq n$, 且 p 和 q 同为 s 的 period, 那么 $\gcd(p, q)$ 也是 s 的 period。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

如果 $u > d$, 那么第一次匹配往右移 d 位显然又是一个匹配, 故 $u = d$, 余下同理。

border 与字符串匹配

Theorem 1.3

对于字符串 s 和 t , 若 $2n \geq m$, 则 s 在 t 中的所有匹配位置构成了一个等差数列。

Proof

只需要讨论匹配 3 次及以上的情况。设第 1, 2 次的距离为 u 、第 2, 3 次匹配的距离为 v , 则 $u + v \leq n$, 且 u, v 同为 s 的 period。故 $d = \gcd(u, v)$ 也为 s 的 period。

如果 $u > d$, 那么第一次匹配往右移 d 位显然又是一个匹配, 故 $u = d$, 余下同理。

不难发现, 如果匹配次数不小于 3, 则这个等差数列的公差就是 s 的最小 period。

border 的结构

Theorem 1.4

字符串 s 的所有长度 $\geq \frac{n}{2}$ 的 border 长度构成了一个等差数列。

border 的结构

Theorem 1.4

字符串 s 的所有长度 $\geq \frac{n}{2}$ 的 border 长度构成了一个等差数列。

Proof

设 s 的最小 period 为 p , 且 $n - p \geq \frac{n}{2}$ 。考虑另一个 period q , 满足 $n - q \geq \frac{n}{2}$, 则 $p + q \leq n$, 故 q 是 p 的倍数。并且显然所有 p 的倍数都是 s 的 period, 于是得证。

LargePS

Theorem 1.5

设 $|s| = |t| = n$, $\text{LargePS}(s, t)$ 表示满足 $p \geq n/2$ 且 $\text{pre}(s, p) = \text{suf}(t, p)$ 的所有 p , 则 $\text{LargePS}(s, t)$ 组成了一个等差数列。

LargePS

Theorem 1.5

设 $|s| = |t| = n$, $\text{LargePS}(s, t)$ 表示满足 $p \geq n/2$ 且 $\text{pre}(s, p) = \text{suf}(t, p)$ 的所有 p , 则 $\text{LargePS}(s, t)$ 组成了一个等差数列。

Proof

设最大元素为 x 。若 $p < x$ 也在 $\text{LargePS}(s, t)$ 中, 则 $\text{pre}(s, p) = \text{suf}(\text{pre}(s, x), p)$, 所以 p 是 $\text{pre}(s, x)$ 的 border。又 $p \geq n/2 \geq x/2$, 由 Theorem 1.4, 这些 p 构成一个等差数列; 再添上首项 x 仍是等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

因此这一段内的 border 长度，是 $\text{LargePS}(a_i, b_i)$ 与区间 $[2^{i-1}, 2^i)$ 的交。由 Theorem 1.5，它是一个等差数列。

border 的分解

Theorem 1.6

字符串 s 的所有 border 长度可以划分成 $\mathcal{O}(\log n)$ 个等差数列。

Proof

设 $2^k \leq n < 2^{k+1}$ 。先考虑 $[2^{i-1}, 2^i)$ ，其中 $1 \leq i \leq k$ 。令 $a_i = \text{pre}(s, 2^i)$ ， $b_i = \text{suf}(s, 2^i)$ 。

对任意 $p \in [2^{i-1}, 2^i)$ ，有 $p \geq |a_i|/2$ 。并且 p 是 s 的 border，当且仅当 $\text{pre}(a_i, p) = \text{suf}(b_i, p)$ ，也就是 $p \in \text{LargePS}(a_i, b_i)$ 。

因此这一段内的 border 长度，是 $\text{LargePS}(a_i, b_i)$ 与区间 $[2^{i-1}, 2^i)$ 的交。由 Theorem 1.5，它是一个等差数列。

最高段 $[2^k, n)$ 中所有长度都至少为 $n/2$ ，由 Theorem 1.4 也是一个等差数列。这样的长度段只有 $\mathcal{O}(\log n)$ 个。

例题

论战捆竹竿

给定 $s \in \Sigma^n$ 和 w 。初始有一根颜色串为 s 的竹竿。

每次取一根新的 s ，选择 $p \in [0, n]$ ，将新竹竿长度为 p 的前缀与当前竹竿长度为 p 的后缀重合；要求重合位置字符相同。

求所有可达总长度中不超过 w 的不同取值个数。

$T = 5$, $n \leq 5 \times 10^5$, $w \leq 10^{18}$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

例题

JOJO

初始 $S = \varepsilon$ 。依次执行 q 次操作：1 $x\ c$ 表示令 $S \leftarrow Sc^x$ ，且保证 $S = \varepsilon$ 或 S 的末字符不是 c ；2 x 表示令 S 回到第 x 次操作后的状态， $x = 0$ 表示空串。

每次操作后输出 $\sum_{i=1}^{|S|} \pi_i \bmod 998244353$ ，其中
 $\pi_i = \max\{p < i \mid \text{pre}(S, p) = \text{suf}(\text{pre}(S, i), p)\}$ 。
 $q \leq 10^5$ ，每个 1 操作中 $x \leq 10^4$ 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM**
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

fail 和完整转移可以用 BFS 求出。根的儿子们的 fail 为根；处理结点 u 的儿子 $v = \text{ch}(u, c)$ 时，令 $\text{fail}(v) = \delta(\text{fail}(u), c)$ 。若 $\text{ch}(u, c)$ 不存在，就令它等于 $\delta(\text{fail}(u), c)$ ，BFS 顺序保证右边已经被求出。

ACAM

给定模式串集合，先把所有模式串插入 Trie。每个结点 u 对应根到 u 的字符串，记为 $\text{str}(u)$ 。

$\text{fail}(u)$ 定义为满足 $\text{str}(v)$ 是 $\text{str}(u)$ 的真后缀的最深结点 v 。这就是 KMP 的失配边在 Trie 上的多模式版本。

把不存在的 Trie 转移也补上以后， $\delta(u, c)$ 表示 $\text{str}(u)c$ 的最长 Trie 后缀。于是扫描文本时，当前状态始终是已经读入前缀的最长 Trie 后缀。

fail 和完整转移可以用 BFS 求出。根的儿子们的 fail 为根；处理结点 u 的儿子 $v = \text{ch}(u, c)$ 时，令 $\text{fail}(v) = \delta(\text{fail}(u), c)$ 。若 $\text{ch}(u, c)$ 不存在，就令它等于 $\delta(\text{fail}(u), c)$ ，BFS 顺序保证右边已经被求出。

扫描文本 t 时，设读完 $t[1..i]$ 后的状态为 u_i 。所有以 i 为右端点的模式串，恰好是 u_i 在 fail 链上的终止结点。

例题

A task for substrings

给定互不相同的模式串集合 $S = \{s_1, s_2, \dots, s_N\}$ 、文本串 t 和 Q 次询问 $[l, r]$ 。

对每次询问，求满足 $l \leq a \leq b \leq r$ 且 $t[a..b] \in S$ 的二元组 (a, b) 数量。

$N, Q \leq 5 \times 10^5$, $|t| \leq 5 \times 10^6$, $\sum |s_i| \leq 10^6$ 。

[题目链接](#)

例题

Modest Substrings

给定整数区间 $[l, r]$ 和长度 n 。在所有长度为 n 的数字串 a 中，最大化满足 $\text{value}(a[i..j]) \in [l, r]$ 的子串数量；有多解时输出字典序最小的 a 。

含前导零的子串不计入答案，但 a 本身允许有前导零。

$1 \leq l \leq r \leq 10^{800}$, $n \leq 2000$ 。

[题目链接](#)

例题

novel

给定 N 个字符串 $S_1, S_2, \dots, S_N$ 及权值 $w_1, w_2, \dots, w_N$ 。设 $f(S, T)$ 为 T 在 S 中的出现次数， $g(S) = \sum_{i=1}^N w_i f(S, S_i)$ 。

对非空串 S ，定义 $h(S) = \max_{1 \leq l \leq r \leq |S|} \frac{g(S[l..r])}{r-l+1}$ 。求所有 $h(S_i[1..j])$ 。

$N \leq 2 \times 10^5$ ， $\sum |S_i| \leq 5 \times 10^6$ ， $\Sigma = \{a, b, c, d\}$ 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关**
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

对 t 的每个位置 i ，记 d_i 为以 i 为中心的最大回文半径，也就是最大的 d_i 使得 $t[i - d_i + 1..i + d_i - 1]$ 是回文串。

Manacher

对原串两端及相邻字符之间插入同一个分隔符，这样原串的奇回文和偶回文都会变成 t 中的奇回文。

对 t 的每个位置 i ，记 d_i 为以 i 为中心的最大回文半径，也就是最大的 d_i 使得 $t[i - d_i + 1..i + d_i - 1]$ 是回文串。

只要知道 d_i ，一个原串区间是不是回文、某个中心的最大偶回文半径、最长回文子串等信息，都可以由中心和半径直接换算回来。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

若 $i \leq R$ ，令 $j = L + R - i$ 为 i 关于 $[L, R]$ 中心的对称点。区间内部的对称性保证 $d_i \geq \min(d_j, R - i + 1)$ ，于是先用这个值作为初始半径。

Manacher

从左到右求 d_i 。算法始终维护当前已经得到的、右端点最大的回文区间 $[L, R]$ 。

若 $i > R$ ，则当前中心还没有被任何已知回文覆盖，只能令 $d_i \leftarrow 1$ 后向两侧暴力扩展。

若 $i \leq R$ ，令 $j = L + R - i$ 为 i 关于 $[L, R]$ 中心的对称点。区间内部的对称性保证 $d_i \geq \min(d_j, R - i + 1)$ ，于是先用这个值作为初始半径。

只有当初始回文恰好顶到右边界 R 时，才可能继续向外扩展；否则 d_i 已经完全由 d_j 决定。

Manacher

每次求出 d_i 后, 若 $i + d_i - 1 > R$, 就用新的回文区间更新 L, R 。

Manacher

每次求出 d_i 后，若 $i + d_i - 1 > R$ ，就用新的回文区间更新 L, R 。

复杂度证明只看暴力比较：一次成功扩展会使右端点 R 至少右移一位；不成功的比较每个中心至多发生一次。

Manacher

每次求出 d_i 后，若 $i + d_i - 1 > R$ ，就用新的回文区间更新 L, R 。

复杂度证明只看暴力比较：一次成功扩展会使右端点 R 至少右移一位；不成功的比较每个中心至多发生一次。

因此所有中心上的总扩展次数是 $\mathcal{O}(|t|)$ ，Manacher 在线性时间内求出所有半径。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

$\text{fail}(u)$ 指向 u 的最长真回文后缀。为了同时处理奇偶长度，建立两个根：奇根长度为 -1 ，偶根长度为 0 ，并令偶根的 fail 指向奇根。

PAM

PAM (Palindromic Tree) 的每个非根结点对应一个本质不同回文子串。结点 u 记录长度 $\text{len}(u)$ ，以及在两端加入同一个字符后的转移。

$\text{fail}(u)$ 指向 u 的最长真回文后缀。为了同时处理奇偶长度，建立两个根：奇根长度为 -1 ，偶根长度为 0 ，并令偶根的 fail 指向奇根。

处理到位置 p 后，指针 last 表示当前前缀的最长回文后缀。沿 last 的 fail 链向上走，恰好枚举当前前缀的所有回文后缀。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移, 那么新前缀的最长回文后缀就是这个转移指向的结点。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移, 那么新前缀的最长回文后缀就是这个转移指向的结点。

否则新建结点 v , 令 $\text{len}(v) = \text{len}(u) + 2$, 并把 u 的 s_p 转移连到 v 。

PAM

向右加入字符 s_p 时, 从 $last$ 开始沿 fail 跳, 直到找到第一个结点 u 满足 $s_{p-\text{len}(u)-1} = s_p$ 。

若 u 已经有字符 s_p 的转移, 那么新前缀的最长回文后缀就是这个转移指向的结点。

否则新建结点 v , 令 $\text{len}(v) = \text{len}(u) + 2$, 并把 u 的 s_p 转移连到 v 。

v 的 fail 也用同样的方法求: 从 $\text{fail}(u)$ 开始继续跳, 找到下一个能被 s_p 包住的回文后缀, 其对应转移就是 $\text{fail}(v)$ 。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

fail 跳转总复杂度均摊线性。

PAM

每加入一个字符至多新建一个结点，因为新增的本质不同回文串只能是当前前缀的最长回文后缀。

fail 跳转总复杂度均摊线性。

若要统计每个回文的出现次数，可以先在每个位置给 *last* 加一，最后按 len 从大到小把计数累加到 fail。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

令 $\text{slink}(u)$ 跳到 fail 链上第一个 diff 改变的位置。于是枚举所有回文后缀时，只需要逐段跳 series link。

PAM 的 series link

对结点 u 定义 $\text{diff}(u) = \text{len}(u) - \text{len}(\text{fail}(u))$ 。

若 $u, \text{fail}(u), \text{fail}^2(u), \dots$ 上连续若干个结点的 diff 相同，那么它们的长度构成同公差等差数列。

令 $\text{slink}(u)$ 跳到 fail 链上第一个 diff 改变的位置。于是枚举所有回文后缀时，只需要逐段跳 series link。

一条 fail 链会被划分成 $\mathcal{O}(\log n)$ 个这样的等差段。

例题

字符串

给定 $s \in \Sigma^n$ 和 q 次询问 (i, r) 。

对每次询问，求满足 $1 \leq l \leq r$ 且
 $s[i..i+l-1] < (s[i+l..i+2l-1])^R$ 的 l 的数量。

$T \leq 5$, $n, q \leq 10^5$, $i+2r-1 \leq n$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

例题

Reverses

给定 $s, t \in \Sigma^n$ 。选择若干个两两不交的区间 $[l_j, r_j]$ ，把 $t[l_j..r_j]$ 各自翻转，要求最终得到 s 。

求区间数量的最小值，并输出任意一组最优区间；若无解，输出 -1 。

$n \leq 5 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

例题

I Love CCPC

称串 x 为 CCPC 串，当且仅当 $|x| \geq 4$, $x_1 = x_2 = x_{|x|}$, 且 $x[3..|x| - 1]$ 是回文串。

给定初始串 s , 随后每次在当前串左端或右端插入一个字符。每次插入后, 求当前串中 CCPC 子串的数量, 按区间计数。

$T \leq 100$, $\sum(|s| + q) \leq 5 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

例题

Palindrome Addicts

初始有一个空队列，支持 q 次操作： $push\ c$ 表示在队尾加入字符 c ， pop 表示弹出队首字符。

每次操作后，输出当前队列从首到尾拼成的字符串中，本质不同回文子串的数量。

$q \leq 10^6$ ， $\Sigma = \{a, \dots, z\}$ ，保证不会对空队列执行 pop 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数**
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

Z 函数

对字符串 s ，定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说， z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

Z 函数

对字符串 s ，定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说， z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$ ，即 $s[L..R] = s[1..R - L + 1]$ 。

若 $i > R$ ，则当前没有可用信息，令 $z_i \leftarrow 0$ 后暴力向右扩展。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

若 $i > R$, 则当前没有可用信息, 令 $z_i \leftarrow 0$ 后暴力向右扩展。

若 $i \leq R$, 令 $j = i - L + 1$ 。区间内部已经和前缀对齐, 所以 $z_i \geq \min(z_j, R - i + 1)$, 先用这个值初始化。

Z 函数

对字符串 s , 定义 $z_i = \text{lcp}(s, s[i..n])$ 。也就是说, z_i 是从位置 i 开始的后缀与整个串的最长公共前缀长度。

从左到右计算 z_i 。算法维护当前右端点最大的、已经确认与前缀相等的区间 $[L, R]$, 即 $s[L..R] = s[1..R - L + 1]$ 。

若 $i > R$, 则当前没有可用信息, 令 $z_i \leftarrow 0$ 后暴力向右扩展。

若 $i \leq R$, 令 $j = i - L + 1$ 。区间内部已经和前缀对齐, 所以 $z_i \geq \min(z_j, R - i + 1)$, 先用这个值初始化。

只有当这个初值正好顶到 R 时, 才继续暴力扩展; 每次成功扩展都会使 R 右移, 因此总复杂度是 $\mathcal{O}(n)$ 。

例题

Prefix of Suffixes

初始 $S = \varepsilon$ 。第 i 次操作给定 S'_i, A_i, B_i , 令
 $S_i = (S'_i + \text{lastans}) \bmod N$, 并将 S_i 追加到 S 。

设当前长度为 n , $z_i = \text{lcp}(S[i..n], S[1..n])$ 。每次操作后输出
 $\sum_{i=1}^n \sum_{j=i}^{i+z_i-1} A_j B_i$, 并令其为新的 lastans。

$N \leq 3 \times 10^5$, $0 \leq S'_i < N$, $1 \leq A_i, B_i \leq 1000$ 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构**
- 7 基本子串结构
- 8 References

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa ，其逆排列为 rk 。

倍增构造的基本状态是：第 k 轮已经知道每个长度 2^k 前缀块的 $rank$ 。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

倍增构造的基本状态是: 第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank, 可以用两次稳定计数排序完成一轮。初始按字符排序, 然后倍增长度, 直到 rank 全部不同。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa ，其逆排列为 rk 。

倍增构造的基本状态是：第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank，可以用两次稳定计数排序完成一轮。初始按字符排序，然后倍增长度，直到 rank 全部不同。

总复杂度 $\mathcal{O}(n \log n)$ 。

后缀数组

后缀数组 (Suffix Array, SA) 把所有后缀按字典序排列。记排序后的数组为 sa , 其逆排列为 rk 。

倍增构造的基本状态是: 第 k 轮已经知道每个长度 2^k 前缀块的 rank。于是长度 2^{k+1} 的块只由有序对 $(c_{k,i}, c_{k,i+2^k})$ 决定。

由于两维 key 都是整数 rank, 可以用两次稳定计数排序完成一轮。初始按字符排序, 然后倍增长度, 直到 rank 全部不同。

总复杂度 $\mathcal{O}(n \log n)$ 。

也存在线性的构造方法。

height 数组

定义 $\text{height}_i = \text{lcp}(s[\text{sa}_{i-1}..n], s[\text{sa}_i..n])$ 。

height 数组

定义 $\text{height}_i = \text{lcp}(s[\text{sa}_{i-1}..n], s[\text{sa}_i..n])$ 。

对任意两个后缀 x, y , 设 $a = \text{rk}_x$, $b = \text{rk}_y$, 且 $a < b$ 。那么
 $\text{lcp}(x, y) = \min_{a < i \leq b} \text{height}_i$ 。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

对状态 u ，记 $\text{len}(u)$ 为该等价类中最长串的长度。这个状态表示的所有串长度恰好是 $(\text{len}(\text{link}(u)), \text{len}(u)]$ ，并且都是最长串的后缀。

SAM 的状态语义

SAM (Suffix Automaton) 是识别 s 的所有子串的最小自动机。它和 Trie / ACAM 最大的区别是：一个状态不对应一个确定串，而对应一整个 endpos 等价类。

对状态 u ，记 $\text{len}(u)$ 为该等价类中最长串的长度。这个状态表示的所有串长度恰好是 $(\text{len}(\text{link}(u)), \text{len}(u)]$ ，并且都是最长串的后缀。

suffix link 指向包含关系下的最长真后缀等价类。把 suffix link 反向后得到 parent tree；一个状态到根的路径，描述了它的最长串的所有后缀等价类。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

若遇到 p 已经有 c 转移到 q ，并且 $\text{len}(q) = \text{len}(p) + 1$ ，则可以直接令 $\text{link}(cur) = q$ 。

SAM 的构造

增量构造时，加入字符 c 后新建状态 cur ，令 $\text{len}(cur) = \text{len}(last) + 1$ ，它代表所有以新位置结尾的后缀中尚未出现的部分。

从 $last$ 沿 suffix link 回退，把所有还没有 c 转移的状态都连向 cur 。若一直回到根都没有遇到已有转移，则 $\text{link}(cur) = root$ 。

若遇到 p 已经有 c 转移到 q ，并且 $\text{len}(q) = \text{len}(p) + 1$ ，则可以直接令 $\text{link}(cur) = q$ 。

否则要复制 q 得到 clone，令它的长度为 $\text{len}(p) + 1$ ，复制 q 的转移和 link，并把从若干祖先指向 q 的 c 转移改到 clone。最后令 q 和 cur 的 link 都指向 clone。

例题

打击复读

给定 $S \in \{A, T, G, C\}^n$ ，每个位置有左权 wl_i 与右权 wr_i 。对任意子串 t ，令 $vl(t)$ 为 t 所有出现左端点的 wl 之和， $vr(t)$ 为所有出现右端点的 wr 之和，贡献为 $vl(t)vr(t)$ 。

初始询问 $\sum_{1 \leq l \leq r \leq n} vl(S[l..r])vr(S[l..r])$ ，随后 m 次单点修改 $wl_u \leftarrow v$ ，每次修改后输出该值，答案对 2^{64} 取模。

$n, m \leq 5 \times 10^5$ ， $0 \leq wl_i, wr_i, v < 2^{64}$ 。

[题目链接](#)

例题

ℳ-function

对字符串 x , 令 $z_i = \text{lcp}(x, x[i..|x|])$, 定义 $\text{zh}(x) = \sum_i z_i$ 。

给定 $s \in \Sigma^n$ 和 q 次询问 $[l, r]$, 每次输出 $\text{zh}(s[l..r])$ 。

$n, q \leq 2 \times 10^5$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

例题

Border 的第五种求法

给定 $s \in \Sigma^n$ 、数组 $f_1, \dots, f_n$ 和 q 次询问 $[l, r]$ 。若字符串 t 在 s 中出现了 c 次，则其价值为 f_c 。

对每次询问，求 $s[l..r]$ 的所有 border 的价值之和。这里 $s[l..r]$ 自身也算作 border。

$n, q \leq 5 \times 10^5$, $1 \leq f_i \leq 10^9$, $\Sigma = \{a, \dots, z\}$ 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构**
- 8 References

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

但是很多内部子串问题会同时询问左端和右端：例如一个正串 parent tree 上的结点与一个反串 parent tree 上的结点，表示的两类串到底交出哪些子串。

基本子串结构

SAM 很擅长刻画一个串的所有后缀信息。

但是很多内部子串问题会同时询问左端和右端：例如一个正串 parent tree 上的结点与一个反串 parent tree 上的结点，表示的两类串到底交出哪些子串。

基本子串结构可以看成把这两个方向放到同一张子串网格中：每个格子是一个子串，每个块是一批在左右上下文中完全同步出现的子串。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

直观地说， $\text{ext}(t)$ 是每一次出现 t 时，前后一定会连带着一起出现的那段最大上下文。

occ 与 ext

令整串为 s 。每个子串 $s[l..r]$ 对应网格中的一个格子 (l, r) ；同一列固定左端，同一行固定右端。

令 $\text{occ}(t)$ 表示 t 在 s 中的出现次数。对子串 t ，定义 $\text{ext}(t)$ 为最长的串 t' ，满足 t 是 t' 的子串且 $\text{occ}(t) = \text{occ}(t')$ 。

直观地说， $\text{ext}(t)$ 是每一次出现 t 时，前后一定会连带着一起出现的那段最大上下文。

若 $\text{ext}(t_1) = \text{ext}(t_2)$ ，就把 t_1, t_2 放在同一个等价类中；网格中同一等价类的一个极大连通部分称为一个块。

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

称块左上角对应的串为这个等价类的代表元，记为 $\text{rep}(C)$ 。它也就是该等价类中最长的串。

块的形状

Lemma 6.1

每个块都是一个上端和左端对齐的阶梯形网格图。也就是说，存在 l_a, l_b, r 和一个单调不降函数 f ，使得这个块中的格子恰为

$$\{(x, y) \mid l_a \leq x \leq l_b, f(x) \leq y \leq r\}.$$

称块左上角对应的串为这个等价类的代表元，记为 $\text{rep}(C)$ 。它也就是该等价类中最长的串。

同一个等价类的所有块形状相同，只是在网格中平移到了不同位置；这样的平移副本数正好是 $\text{occ}(\text{rep}(C))$ 。

abbab 的基本块

$r = 5$	<i>abbab</i>	<i>bbab</i>	<i>bab</i>	<i>ab</i>	<i>b</i>
$r = 4$	<i>abba</i>	<i>bba</i>	<i>ba</i>	<i>a</i>	
$r = 3$	<i>abb</i>	<i>bb</i>	<i>b</i>		
$r = 2$	<i>ab</i>	<i>b</i>			
$r = 1$	<i>a</i>				
	$l = 1$	$l = 2$	$l = 3$	$l = 4$	$l = 5$

ababaca 的基本块

$r = 7$	<i>ababaca</i>	<i>babaca</i>	<i>abaca</i>	<i>baca</i>	<i>aca</i>	<i>ca</i>	<i>a</i>
$r = 6$	<i>ababac</i>	<i>babac</i>	<i>abac</i>	<i>bac</i>	<i>ac</i>	<i>c</i>	
$r = 5$	<i>ababa</i>	<i>baba</i>	<i>aba</i>	<i>ba</i>	<i>a</i>		
$r = 4$	<i>abab</i>	<i>bab</i>	<i>ab</i>	<i>b</i>			
$r = 3$	<i>aba</i>	<i>ba</i>	<i>a</i>				
$r = 2$	<i>ab</i>	<i>b</i>					
$r = 1$	<i>a</i>						
	$l = 1$	$l = 2$	$l = 3$	$l = 4$	$l = 5$	$l = 6$	$l = 7$

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地，若 $t = s[l..r]$ 是代表元，则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数，除非已经碰到边界。

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地，若 $t = s[l..r]$ 是代表元，则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数，除非已经碰到边界。

块中的每一行都对应正串后缀树或 SAM 的一个状态；块中的每一列都对应反串后缀树或 SAM 的一个状态。因此块同时刻画了“共同右上下文”和“共同左上下文”。

结构信息

代表元 $\text{rep}(C)$ 可以由左右两侧都不能在保持出现次数的前提下继续扩张来识别。等价地，若 $t = s[l..r]$ 是代表元，则向左扩到 $s[l-1..r]$ 或向右扩到 $s[l..r+1]$ 都会改变出现次数，除非已经碰到边界。

块中的每一行都对应正串后缀树或 SAM 的一个状态；块中的每一列都对应反串后缀树或 SAM 的一个状态。因此块同时刻画了“共同右上下文”和“共同左上下文”。

BASS 保存两棵树、每个等价类的代表块形状，以及把任意子串定位到其等价类和块内位置的数据结构。所有代表块的周长总和是 $O(n)$ ，所以整套结构可以线性规模表示。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

如果直接在所有 $\mathcal{O}(n^2)$ 个格子上做 DP，状态数太大；但同一个块内的子串拥有相同的 ext 和相同的出现次数，连续删左或删右在块内只是在阶梯形中平移。

用于删左删右的压缩

在子串网格中，删掉左端点对应从 (l, r) 走到 $(l+1, r)$ ，删掉右端点对应从 (l, r) 走到 $(l, r-1)$ 。

如果直接在所有 $\mathcal{O}(n^2)$ 个格子上做 DP，状态数太大；但同一个块内的子串拥有相同的 `ext` 和相同的出现次数，连续删左或删右在块内只是在阶梯形中平移。

因而可以把每个块压成一个结点，边只出现在相邻块之间。块内部的路径数、出口位置和 `occ` 的幂次由阶梯边界一次性计算。

例题

子串统计

给定字符串 S 。从 $T_0 = S$ 出发，每次删除当前串的首字符或尾字符，直到长度为 1。

对一条删除序列 $T_0, T_1, \dots, T_{n-1}$ ，定义贡献为 $\prod_{i=1}^{n-1} \text{occ}_S(T_i)$ 。
求所有删除序列贡献之和，对 998244353 取模。

$1 \leq |S| \leq 10^5$ 。

[题目链接](#)

- 1 记号
- 2 border 相关
- 3 ACAM
- 4 回文串相关
- 5 Z 函数
- 6 后缀数据结构
- 7 基本子串结构
- 8 References

References I

- [CDM⁺23] Jingbang Chen, Jiangqi Dai, Qiuyang Mang, Qingyu Shi, and Tingqiang Xu. Nearly Optimal Internal Dictionary Matching, 2023.
URL: <https://arxiv.org/abs/2312.11873>, arXiv:2312.11873.
- [Chen19] 陈孙立.
子串周期查询问题的相关算法及其应用.
In 国家集训队 2019 论文集. 中国计算机学会, 2019.
- [Cheng23] 程思元.
字符串 1, 2023.
APIO2023 课件.
- [com20] command_block.
Border 理论小记, 2020.
URL: <https://www.luogu.com/article/ds5cz0sg>.
- [ix3521] ix35.
NOI 一轮复习 II: 字符串, 2021.
URL: <https://www.luogu.com/article/e08ebmz6>.
- [ix3523] ix35.
字符串技术巡礼, 2023.
URL: <https://www.luogu.com/article/xz8mjesh>.
- [Jin17] 金策.
字符串算法选讲, 2017.
WC2017 课件.
- [OI 26] OI Wiki.
字符串, 2026.
URL: <https://oi-wiki.org/string/>.

References II

- [Xu23a] 徐安矣.
浅谈回文串问题的相关算法及其应用.
In 国家集训队 2023 论文集. 中国计算机学会, 2023.
- [Xu23b] 许庭强.
一类基础子串数据结构.
In 国家集训队 2023 论文集. 中国计算机学会, 2023.
- [Xu23c] 许庭强.
字符串 2, 2023.
APIO2023 课件.

致谢

谢谢大家！